

Documentación de seguridad y resiliencia

Matías Martínez · Entrega Final · Ecosistema: n8n + Airtable + OpenAI + WhatsApp Cloud API

1. Minimización de datos

- **Solo los campos necesarios viajan entre servicios:** a OpenAI se le envían únicamente las directrices de marca y la idea semilla (nunca datos de clientes, correos ni identificadores); a WhatsApp solo el texto del post aprobado.
- **Credenciales fuera del flujo:** las llaves (token de Airtable, API key de OpenAI, token de Meta) viven en el gestor de credenciales de n8n, nunca escritas en los nodos. El JSON del flujo publicado en este repositorio referencia credenciales por nombre, sin exponer ningún secreto.
- **Tokens con alcance mínimo:** el token de Airtable solo tiene permisos de lectura/escritura de registros y lectura de esquema sobre esta única base (si se filtrara, no expone ninguna otra base); el token de WhatsApp es de un usuario del sistema con los permisos justos de mensajería.
- **Datos personales sanitizados en la documentación:** el número de teléfono destinatario está enmascarado (569XXXXXXXXX) en el JSON público y en este documento.
- La instancia de n8n es **auto-hospedada** en un servidor propio con HTTPS: los datos del pipeline no pasan por orquestadores de terceros.

2. Rutas de error (resiliencia)

El sistema asume que las APIs fallan y que los datos llegan incompletos. Tres rutas de error escriben en la tabla **Log de errores** de Airtable, siempre con el vínculo al registro afectado, el nodo de origen, el detalle devuelto por la API y la fecha:

Punto de fallo	Mecanismo	Qué pasa
Dato faltante (idea semilla vacía) — el "camino infeliz"	Nodo de validación ¿Idea válida? antes de la IA	El registro inválido se desvía al log sin consumir tokens de IA; el resto del lote continúa normal.
Fallo de la API de OpenAI (caída, 429, timeout)	Salida de error del nodo de IA (continueErrorOutput)	Se registra el error con su detalle; el flujo no se congela en rojo y las demás piezas siguen su curso.
Fallo de la API de WhatsApp (sesión vencida, token inválido)	Salida de error del nodo de publicación	Se registra el error; la pieza queda "En revisión" con su aprobación intacta, lista para reintentarse en la siguiente ejecución.

La **tasa de errores** resultante es visible en el dashboard (tabla KPIs), de modo que el deterioro de cualquier API externa se detecta mirando un número, no revisando ejecuciones una por una.

3. Human-in-the-loop (puntos de validación humana)

- **Pausa obligatoria:** el flujo escribe el borrador y deja la pieza "En revisión". No existe ningún camino en el flujo que publique sin pasar por el filtro.
- **Filtro final booleano:** la rama de publicación exige AND({Aprobado}=1, {Estado}='En revisión') en la búsqueda y verifica el checkbox otra vez en un nodo IF. Doble compuerta, cero "efecto metrallera".
- **Rechazo explícito:** el humano puede marcar Estado='Rechazado'; la pieza queda archivada con su marca temporal y visible en el dashboard, sin publicarse jamás.

4. Prevención de bucles infinitos y comparaciones de tipo

- **Los estados actúan como candados:** el flujo solo toma "Generando" (que al procesarse pasa a "En revisión") y solo publica "En revisión + aprobado" (que pasa a "Publicado"). Ningún estado vuelve atrás automáticamente → una pieza no puede procesarse dos veces ni entrar en bucle.
- **Comparaciones con tipos correctos:** el checkbox se evalúa como booleano (operador "is true", no comparación de texto); los estados se comparan como texto exacto contra las opciones del campo de selección única; los vínculos se pasan como arreglos de IDs de registro.
- **Trigger acotado:** programado cada 15 minutos con búsquedas filtradas por fórmula — sin webhooks abiertos al público y sin consumo de operaciones sobre registros que no corresponden.